

LISTAS EM PYTHON

Sumário



TÓPICOS DA APRESENTAÇÃO

- 01 — O Que São Listas?
- 02 — Armazenamento de Múltiplos Valores
- 03 — Sequencial × Encadeada
- 04 — Comandos de Armazenamento
- 05 — Acesso por Índice
- 06 — Percorrendo Listas
- 07 — Aplicações Práticas
- 08 — Fontes e Referências

O Que São Listas?

Uma lista em Python é uma estrutura de dados que armazena múltiplos valores em uma única variável. Principais características:

- Aceita diferentes tipos de dados (inteiros, strings, floats, etc.)
- É mutável — seus elementos podem ser alterados
- Criada com colchetes [] ou com a função list()
- Mantém a ordem dos elementos inseridos



```
2
3  nome = "Amanda"
4
5  #exemplo SEM compreensão de lista
6  lista = []
7
8  for i in nome:
9      lista.append(i)
10 print(lista)
11
12 #saída: ['A', 'm', 'a', 'n', 'd', 'a']
13
14
15 #exemplo COM compreensão de lista
16
17 lista = [ i for i in nome]
18 print(lista)
19
20 #saída: ['A', 'm', 'a', 'n', 'd', 'a']
```

Armazenamento de Múltiplos Valores

ORGANIZAÇÃO POR ÍNDICES

Listas em Python organizam elementos sequencialmente, onde cada valor recebe um índice numérico automático. Os índices começam sempre em 0 (zero), facilitando o acesso preciso a qualquer elemento. Essa estrutura permite armazenar e gerenciar múltiplos valores de forma eficiente, evitando a criação de diversas variáveis separadas. Exemplo: `lista = ["A", "B", "C"]` → `lista[0] = "A"`, `lista[1] = "B"`, `lista[2] = "C"`.

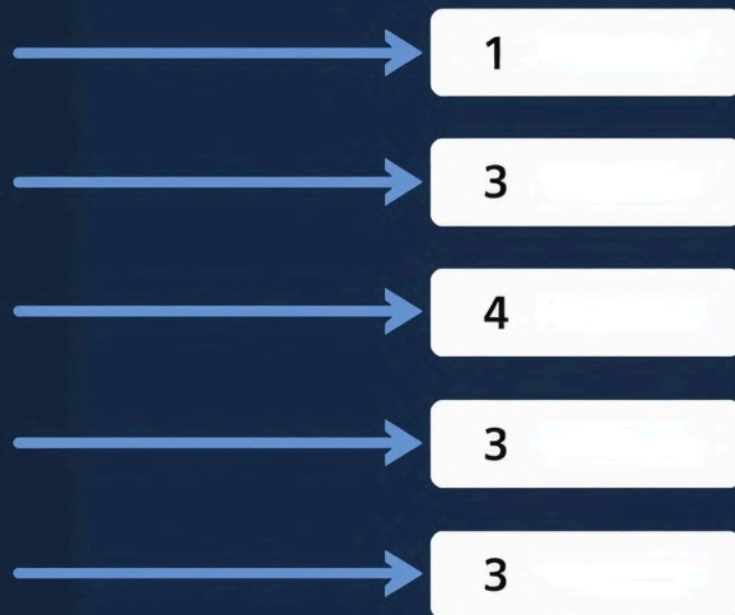


Sequencial X Encadeada

Sequencial Array

0	0	2	2
0	1	1	2
0	1	2	3
0	1	2	4
0	1	2	3

Linked List



COMPARANDO ESTRUTURAS DE LISTAS

Listas Python são arrays dinâmicos que permitem acesso rápido aos elementos via índice, com redimensionamento automático em memória contígua. Listas Encadeadas armazenam elementos em nós conectados por ponteiros, sem alocação contígua. Principais diferenças: acesso por índice ($O(1)$ em arrays vs. $O(n)$ em encadeadas), inserção no meio ($O(n)$ vs. $O(1)$ com ponteiro), e uso de memória (menor em arrays vs. overhead de ponteiros em encadeadas). Em Python, a estrutura nativa `list` é um array dinâmico, ideal para a maioria dos casos de uso.

COMANDOS DE ARMAZENAMENTO

`append()` — Adiciona um elemento ao final da lista.

Exemplo: `lista.append(10)` → `[1, 2, 3, 10]`

`insert()` — Adiciona um elemento em uma posição específica.

Exemplo: `lista.insert(1, 99)` → `[1, 99, 2, 3]`

Ambo os métodos modificam a lista original (mutabilidade). O `append()` é mais eficiente para inserções ao final, enquanto o `insert()` permite controle preciso da posição do novo elemento.



`append()` `insert()` em Python



COMANDO	FUNÇÃO
Pop	Remove um item da lista, podendo ser um específico ou o último.
Remove	Usado para remover itens específicos.
Sort	Colocar a lista em ordem.
Len	Serve para saber o comprimento da lista.
Reverse	Quando usado em conjunto com o sort coloca a lista em ordem decrescente, como por exemplo: “lista.sort(reverse=True)”.
Count	Conta quantas vezes determinado elemento apareceu

Acesso por Índice

NAVEGANDO NOS ELEMENTOS

Índices Positivos: 0, 1, 2, ... acessam elementos do início da lista. O primeiro elemento sempre tem índice 0.

Índices Negativos: -1, -2, ... acessam elementos a partir do fim. O último elemento tem índice -1.

Exemplo:

lista = ['a', 'b', 'c', 'd']

lista[0] → 'a'

lista[2] → 'c'

lista[-1] → 'd'

lista[-2] → 'c'

```
out << "Enter rows and columns for second matrix: ";
cin >> r2 >> c2;

// Entering elements of first matrix.
out << endl << "Enter elements of matrix 1:" << endl;
for(i = 0; i < r1; ++i)
    for(j = 0; j < c1; ++j)
    {
        out << "Enter element a" << i + 1 << j + 1 << " : ";
        cin >> a[i][j];
    }

// Entering elements of second matrix.
out << "Enter elements of matrix 2:" << endl;
for(i = 0; i < r2; ++i)
    for(j = 0; j < c2; ++j)
    {
        out << "Enter element b" << i + 1 << j + 1 << " : ";
        cin >> b[i][j];
    }
```

INTERAÇÃO COM LAÇOS

Laço for — mais comum e legível, percorre automaticamente cada elemento da lista sem necessidade de controlar o índice manualmente.

Laço while — oferece controle manual do índice, útil quando a condição de parada é personalizada ou dinâmica.

Ambos permitem acessar, processar e transformar os elementos de uma lista de forma eficiente.

Percorrendo Listas



```
resp_iter = self.stub.GetResponse()
statuses = {}
for data in resp_iter:
    status = Status(
        status_id=data.id, name=data.name
    )
    statuses[status.name] = status
```



Aplicações Práticas

LISTAS EM SISTEMAS DO MUNDO REAL

Trello: listas de tarefas organizadas por quadros e colunas, permitindo gerenciamento visual de projetos.

Spotify: playlists são listas de reprodução que armazenam e ordenam músicas dinamicamente.

WhatsApp: listas de mensagens e contatos gerenciadas sequencialmente em tempo real.

Desenvolvimento Web: manipulação de dados em interfaces, APIs e renderização dinâmica de conteúdo.



Tuplas em python

- Estrutura de armazenamento que, assim como a lista, armazena múltiplos elementos, indicados por índices. Esse tipo de estrutura não apresenta flexibilidade, permanecendo imutável. No Python, uma tupla é frequentemente representada por parênteses.

```
>>> minha_tupla = ([1,2,3],['a','b','c'])
>>> minha_tupla
([1, 2, 3], ['a', 'b', 'c'])
>>>
>>> id(minha_tupla)
140197171822408
>>>
>>> minha_tupla[0].append(4)
>>> minha_tupla
([1, 2, 3, 4], ['a', 'b', 'c'])
>>>
>>> id(minha_tupla)
140197171822408
```

Dicionário em Python

é uma estrutura de dados embutida que armazena informações no formato de **chave e valor**, funcionando como uma memória associativa super rápida onde cada chave mapeia para um valor específico

```
agenda_melhor1 = {
    'lucas (professor)': {
        'email': 'lucas.goncalves@faculdadeimpacta.com.br',
        'telefones': [11999888999, 1177788899]
    },
    'maria' : {
        'email': 'maria.aparecida@exemplo.com',
        'telefones': [84999777444]
    },
    'lucas': {
        'telefones': [1177788899]
    }
}
```



FONTES E REFERÊNCIAS